

Projeto e Análise de Algoritmos

Técnicas de projeto

Thiago Pinheiro de Araújo

EMAp/FGV

2025.2

Apresentar algumas **técnicas de projeto** para desenvolver algoritmos e aplicá-las à alguns problemas de exemplo.

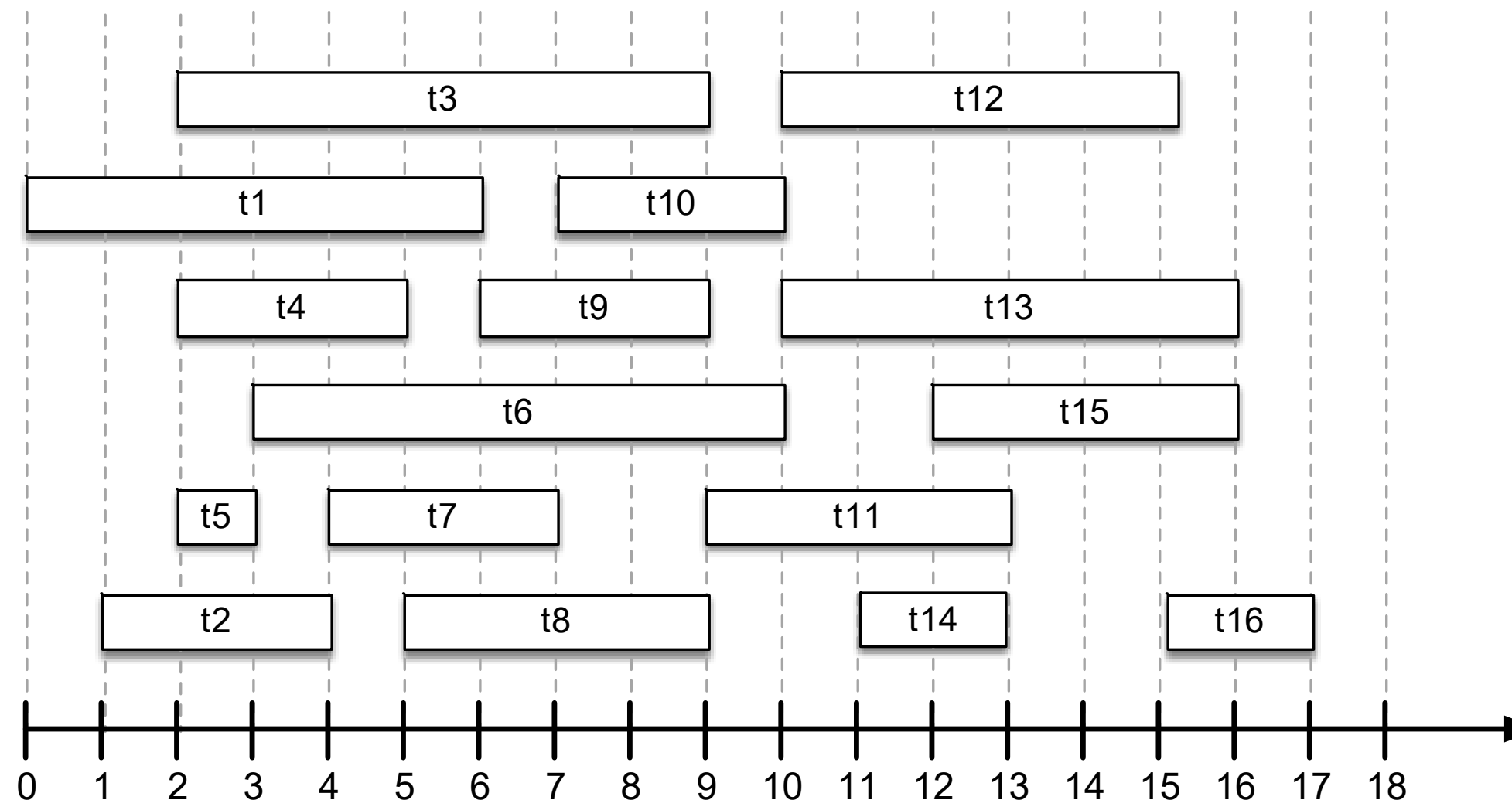
Método guloso (greedy)

Método guloso

- O **método guloso** (*greedy*) é um paradigma bastante utilizado para projeto de algoritmos.
- A estratégia consiste em escolher a cada iteração a opção com maior valor, e avaliar se deve ser adicionada ao resultado final.
- Seguindo essa abordagem as opções precisam ser ordenadas seguindo algum critério.
- Costuma ser simples e eficiente, porém nem todo problema pode ser resolvido através dessa abordagem.

Método guloso

- **Problema (agendamento de tarefas):** Dado o conjunto de tarefas $T = \{t_1, t_2, \dots, t_n\}$ com n elementos, cada uma com um tempo de início $start[tk]$ e um tempo de término $end[tk]$, encontre o maior subconjunto de tarefas que pode ser alocado sem sobreposição temporal.



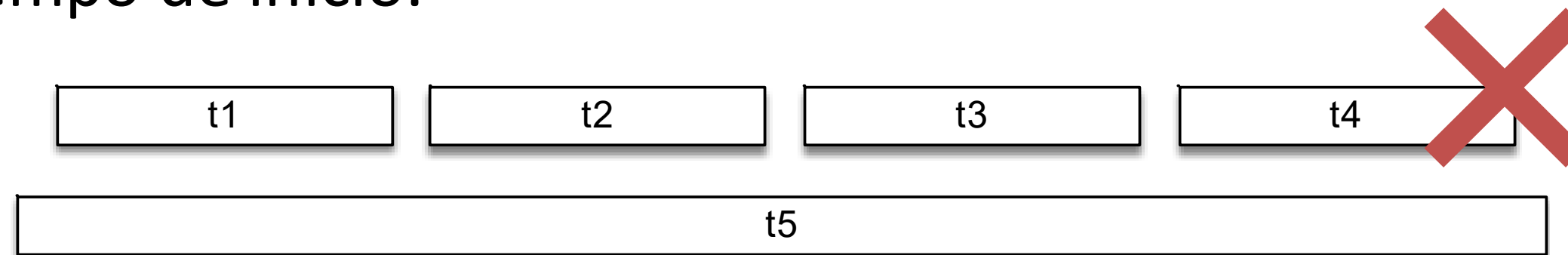
Método guloso

- Vamos projetar um algoritmo para esse problema utilizando o método guloso.
- Pergunta 1: Quais serão as opções a serem avaliadas à cada iteração?
 - Conjunto de tarefas que ainda não foi alocada ou descartada.
- Pergunta 2: Qual critério iremos utilizar para ordenar as opções?
 - Tempo de início?
 - Menor duração?
 - Menor número de conflitos?
 - Tempo de término?

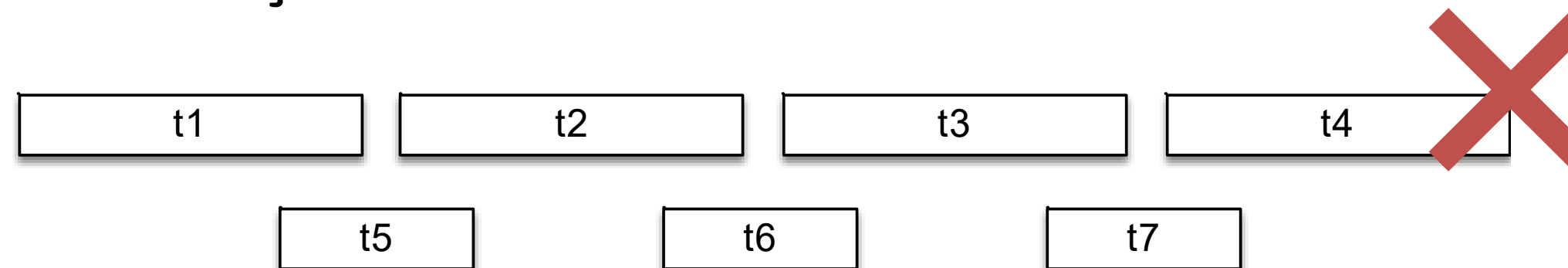
Método guloso

- Sugestão: vamos avaliar cada critério tentando construir ao menos um cenário que demonstre que o critério gera um resultado não-ótimo.

- Critério pelo tempo de início:



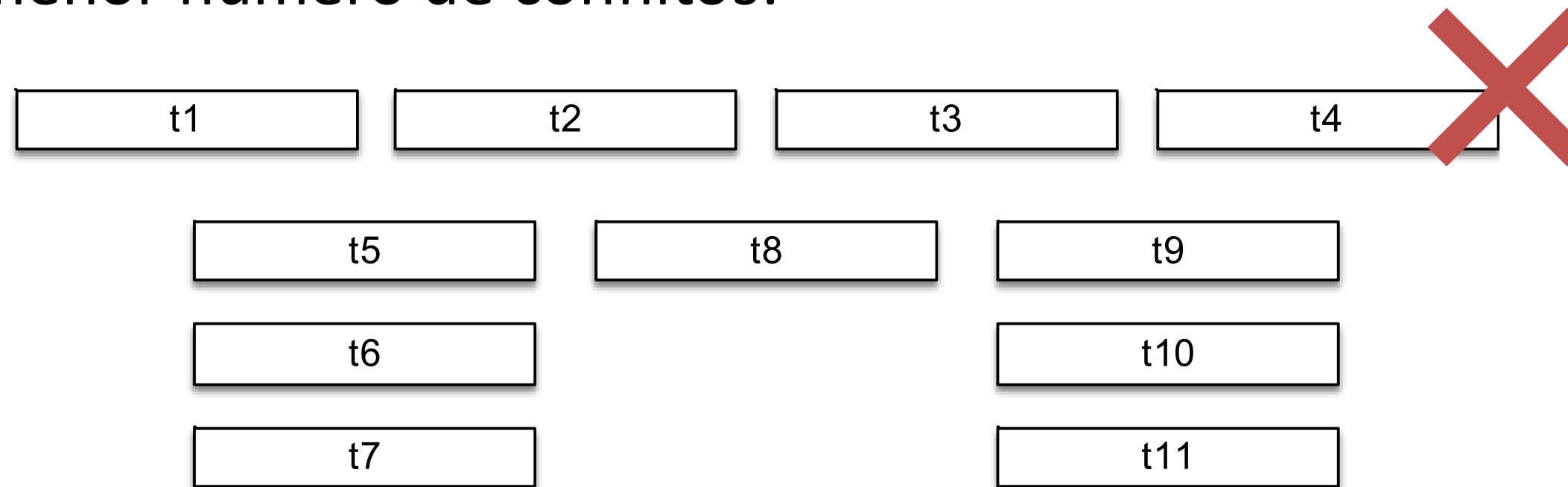
- Critério pela menor duração:



Método guloso

- Sugestão: vamos avaliar cada critério tentando construir ao menos um cenário que demonstre que o critério gera um resultado não-ótimo.

- Critério pelo menor número de conflitos:



- Critério pelo tempo de término.



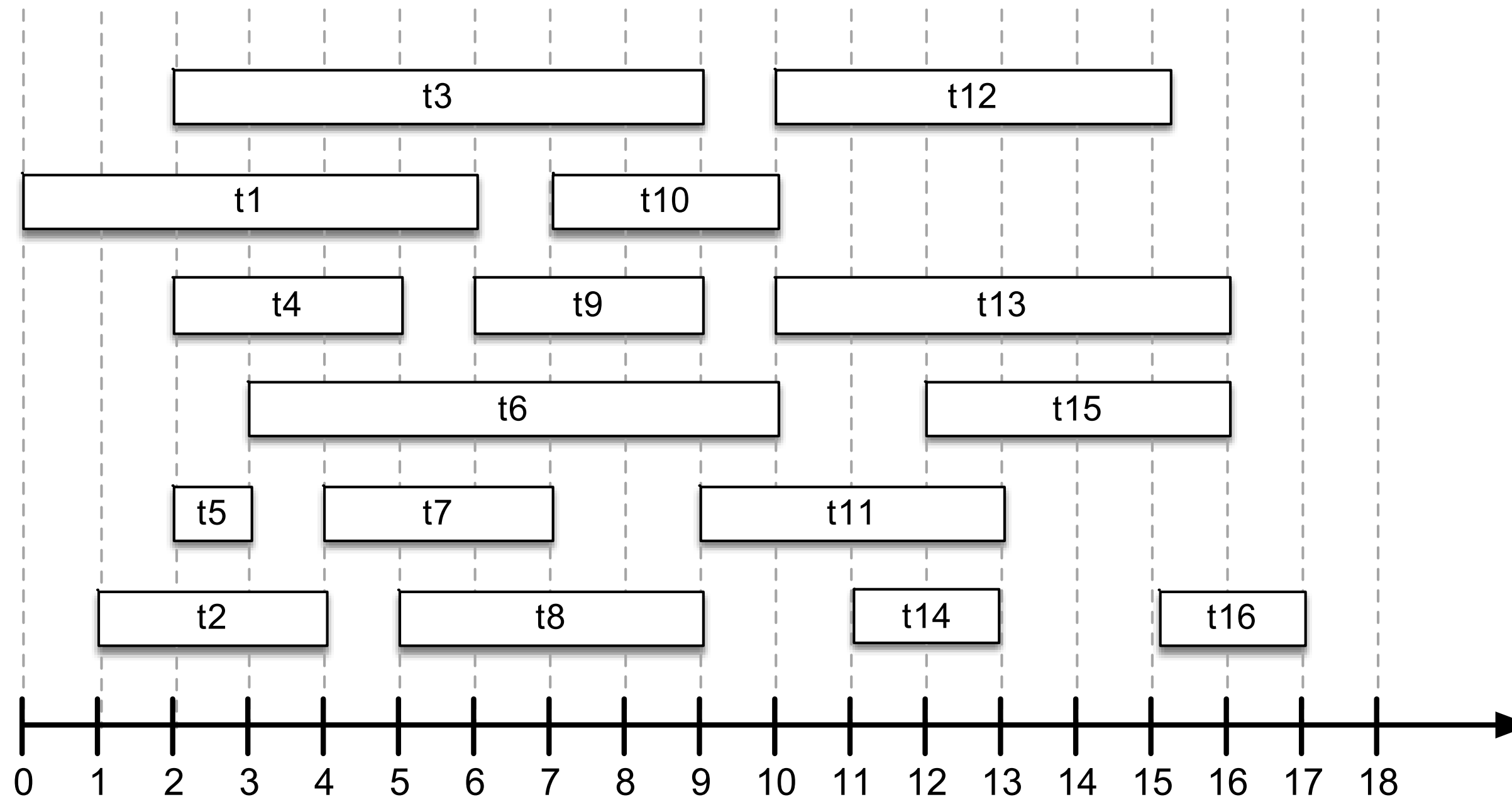
Método guloso

- Ideia geral:
 - Ordene as tarefas pelo tempo de término em uma lista T .
 - Crie um conjunto T_a para armazenar as tarefas a serem alocadas.
 - Insira a primeira tarefa da lista t_0 em T_a e defina $t_{prev} = t_0$.
 - Para cada tarefa t_k em T :
 - Se $start[tk] \geq end[tprev]$
 - Adicione t_k em T_a .
 - Defina $t_{prev} = tk$.
 - Retorne T_a .

Complexidade: $\theta(n \log n)$

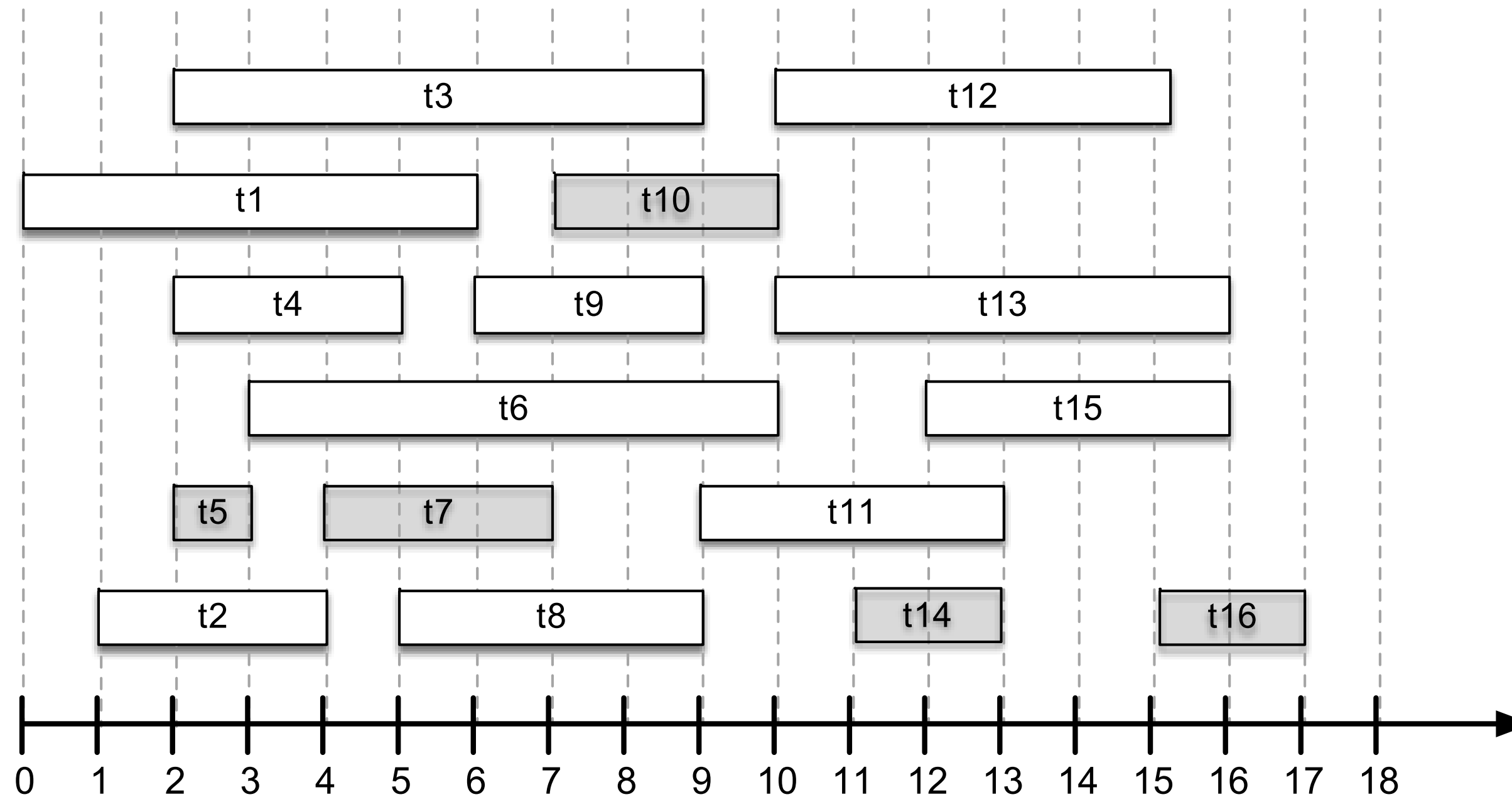
Método guloso

- Solução para o nosso exemplo:



Método guloso

- Solução para o nosso exemplo:

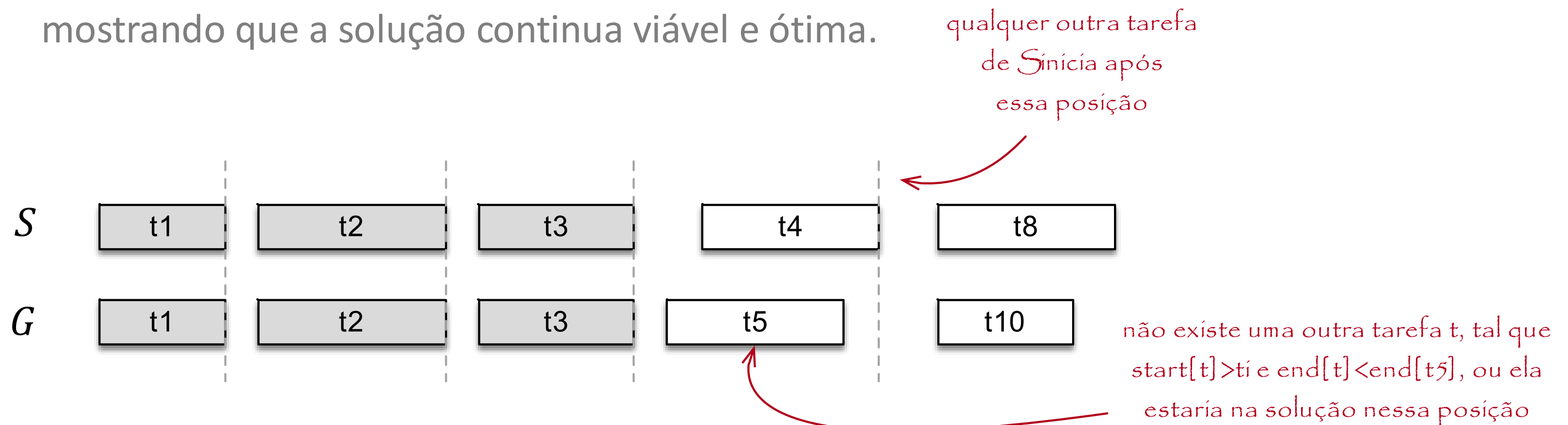


Método guloso

- Como provar que a **solução é ótima**?
- Escolha gulosa: uma solução ótima global pode ser atingida realizando uma sequência de escolhas locais ótimas (gulosas)
 - A escolha local não considera o resultado das escolhas posteriores, e produz um sub-problema contendo um número menor de elementos.
 - A definição do critério de escolha nos auxilia à organizar os elementos de forma que o algoritmo seja eficiente.
- Subestrutura ótima: ocorre quando uma solução ótima de um problema apresenta dentro dela soluções ótimas para sub-problemas.

Método guloso

- Precisamos agora demonstrar que uma solução ótima para um subproblema, ao ser combinada com uma escolha gulosa, produz uma solução ótima
 - Uma estratégia comum é a troca de elementos (*swap argument*):
 - Considere que temos uma solução ótima S , e a solução gulosa G .
 - Troque cada escolha de S por uma escolha de G mostrando que a solução continua viável e ótima.



Método guloso

- **Problema (mochila fracionária):** dado um conjunto de itens $I = \{1, 2, 3, \dots, n\}$ em que cada item $i \in I$ tem um peso w_i e um valor v_i , e uma mochila com capacidade de peso W , encontre o subconjunto $S \subseteq I$ tal que $\sum_{i \in S} \alpha_i w_i \leq W$ e $\sum_{i \in S} \alpha_i v_i$ seja máximo, considerando que $0 < \alpha_k \leq 1$.

- Exemplo:
 - $W = 9$.
 - A escolha $\{1, 2, 3\}$ tem peso 8, valor 12 e cabe na mochila.
 - A escolha $\{3, 5\}$ tem peso 11, valor 14 e não cabe na mochila.

Item	Peso	Valor
1	1	2
2	3	4
3	4	6
4	5	10
5	7	8

- Mais alguns exemplos:
 - $W = 9$.
 - A escolha $\{3_{50\%}, 5_{100\%}\}$ tem peso 9, valor 11 e cabe na mochila.
 - A escolha $\{1_{100\%}, 3_{75\%}, 4_{100\%}\}$ tem peso 9, valor 16.5 e cabe na mochila.

Item	Peso	Valor
1	1	2
2	3	4
3	4	6
4	5	10
5	7	8

Método guloso

- Seria possível criar um algoritmo guloso capaz de encontrar uma solução ótima para esse problema?
- Pergunta 1: Quais serão as opções a serem avaliadas à cada iteração?
 - Itens (ou fragmentos dos mesmos) que ainda não foram adicionados ou descartados.
- Pergunta 2: Qual critério iremos utilizar para ordenar as opções?
 - Menor peso?
 - Menor valor?
 - Maior razão valor/peso?

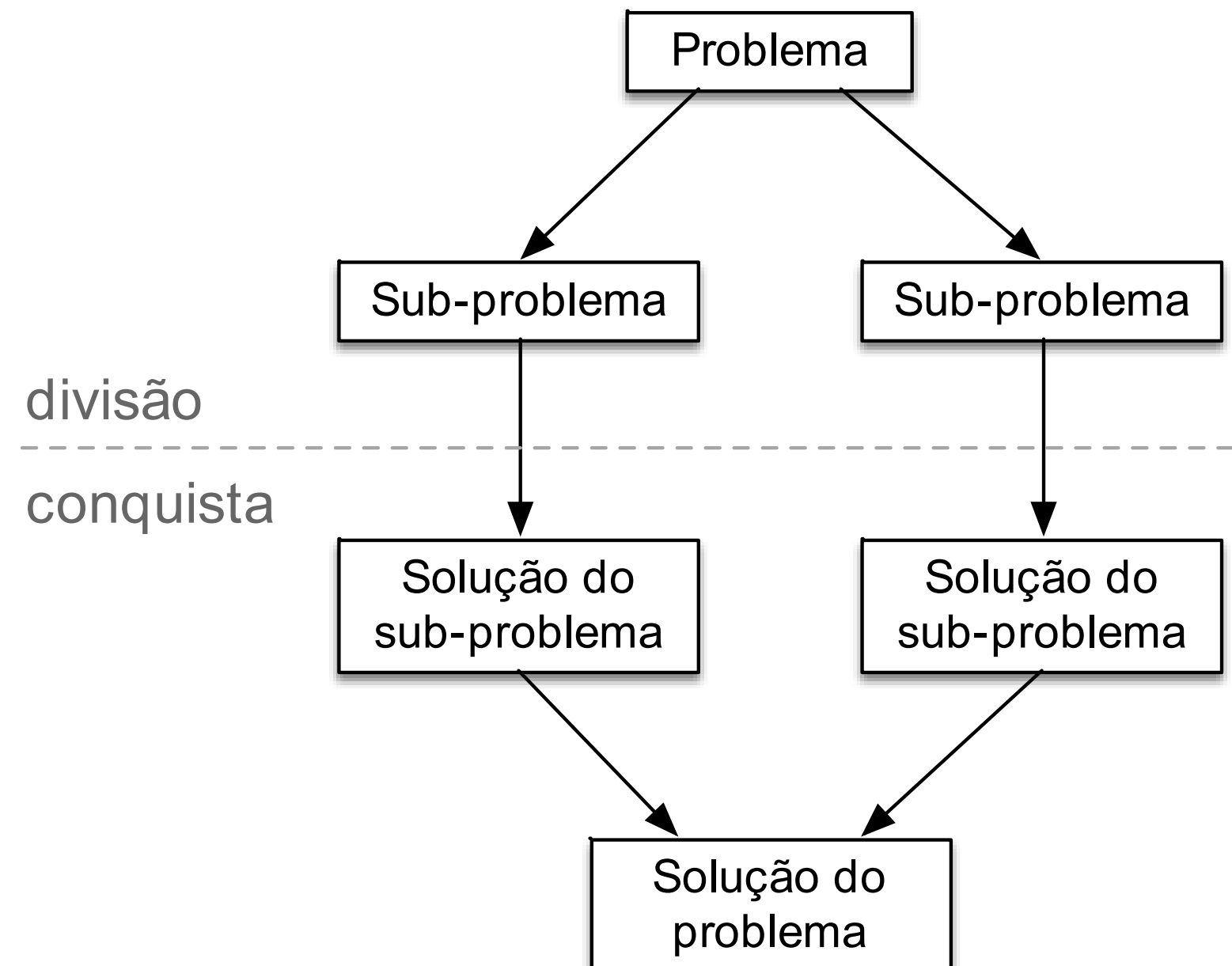
Método guloso

- $mochila(I, v, w, n, W)$
 - Ordene os itens pela razão valor/peso
 - $C=W$ e $i=1$
 - Crie um vetor $M[1..n]$ e inicialize com 0
 - Enquanto $i \leq n$ e $C \geq w_i$:
 - $M[i] = 1$
 - $C = C - w_i$
 - $i += 1$
 - Se $i \leq n$:
 - $M[i] = C/w_i$
 - Retorna M

Complexidade: $\theta(n \log n)$

Dividir e Conquistar

- O paradigma de **divisão e conquista** é composto por três etapas:
- Dividir o problema em um conjunto de sub-problemas menores.
- Resolver cada sub-problema recursivamente.
- Combinar os resultados dos sub-problemas gerando a solução.



- **Problema (contagem de inversões):** dado uma sequência com n números, calcule o número de inversões necessário para torná-la ordenada.
- Como exemplo, considere a sequência a seguir:
 - $A = \{3, 7, 2, 9, 5\}$
- O número total de inversões é 4:
 - $(7, 2), (3, 2), (9, 5), (7, 5)$

Dividir e conquistar

- A solução por força bruta seria verificar todos os pares, exigindo $\theta(n^2)$ operações
 - Acreditamos que seja possível encontrar uma solução mais eficiente.
- A solução baseada em dividir e conquistar deverá definir estratégias para:
 - Dividir a instância do problema em grupos menores.
 - Resolver cada instância individualmente calculando o número de inversões nela.
 - Combinar os resultados encontrando o número de inversões total entre as instâncias combinadas.

Dividir e conquistar

- Podemos dividir a sequência em dois grupos com aproximadamente o mesmo tamanho:
 - 1 até $n/2$.
 - $n/2 + 1$ até n .

2	5	4	1	9	7	6	11	3	12	8	10
---	---	---	---	---	---	---	----	---	----	---	----

2	5	4	1	9	7	6	11	3	12	8	10
---	---	---	---	---	---	---	----	---	----	---	----

- Essa operação será executada em tempo constante – $O(1)$

- A **estratégia de resolução** deve contar o número de inversões em cada grupo:

2	5	4	1	9	7	6	11	3	12	8	10
---	---	---	---	---	---	---	----	---	----	---	----

2	5	4	1	9	7	6	11	3	12	8	10
---	---	---	---	---	---	---	----	---	----	---	----

5 inversões

$(2,1), (5,4), (5,1), (4,1), (9,7)$

6 inversões

$(6,3), (11,3), (11,8), (11,10), (12,8), (12,10)$

- Esse resultado pode ser atingido executando o algoritmo recursivamente – $2T(n/2)$

- A estratégia para combinar os resultados deve contar o número de inversões entre grupos:

2	5	4	1	9	7	6	11	3	12	8	10
---	---	---	---	---	---	---	----	---	----	---	----

2	5	4	1	9	7	6	11	3	12	8	10
---	---	---	---	---	---	---	----	---	----	---	----

5 inversões

6 inversões

+ 7 inversões ao combinar

(5,3), (4,3), (9,6), (9,3), (9,8), (7,6), (7,3)

Total: 18 inversões

- No entanto, como combinar os grupos?
 - **Ideia:** ordenar cada segmento, de forma que a contagem de inversões entre grupos possa ser realizada em tempo linear – $O(n)$.

1	2	4	5	7	9	3	6	8	10	11	12
---	---	---	---	---	---	---	---	---	----	----	----

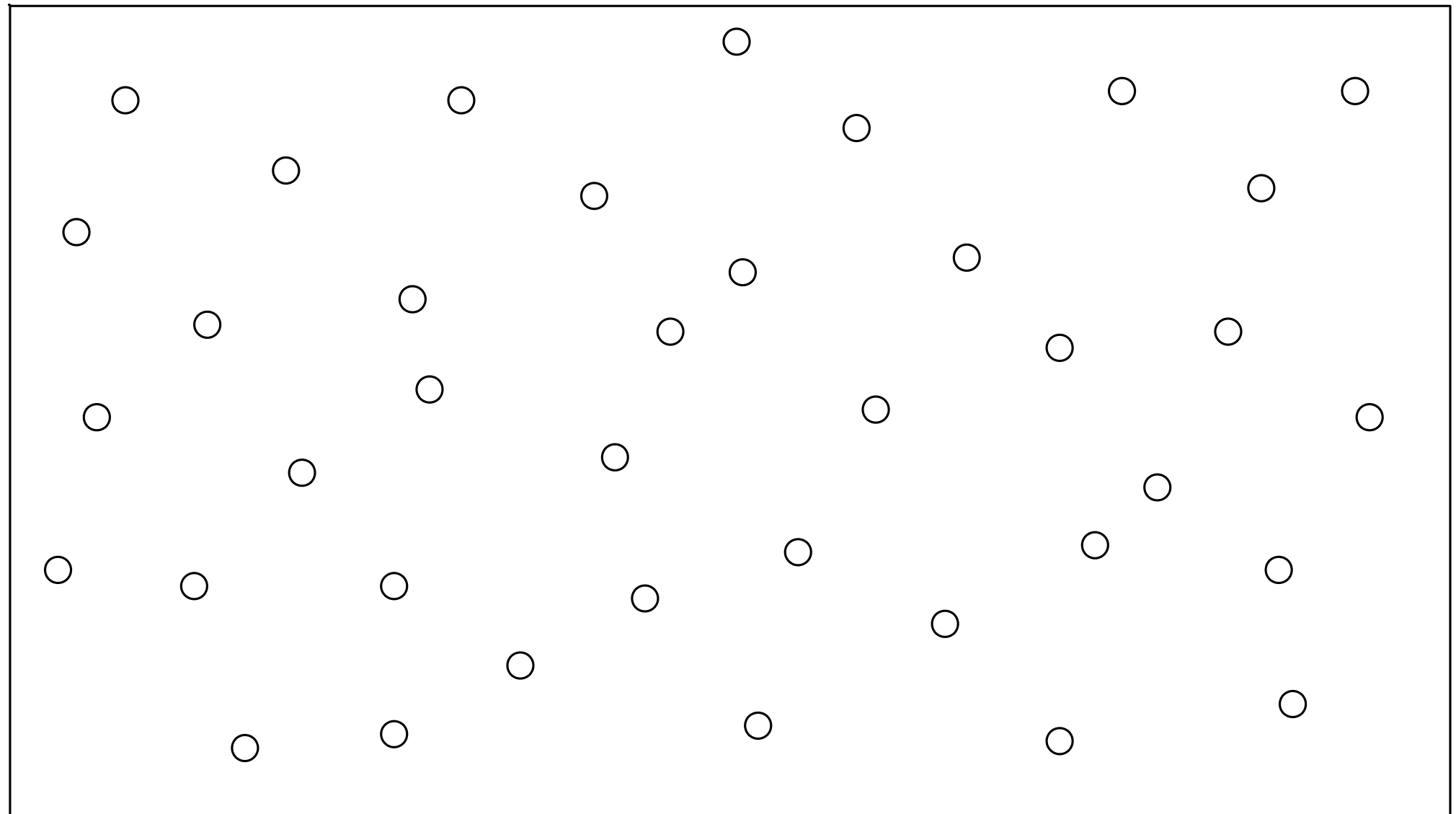
- Considere as sequências como L e R , e defina $i = 0$
 - Para cada elemento a_j de R :
 - Incremente i até $L[i] > a_j$
 - O número de inversões de a_j será igual à $|L| - i$
- Observe que ordenar os conjuntos individualmente não altera as inversões entre eles.

- *CountInversions(A)*:
 - Se $|A| = 1$
 - Retorne 0 inversões
 - Divida a lista A em duas metades L e R com aproximadamente o mesmo tamanho
 - $i_l = \text{CountInversions}(L)$
 - $i_r = \text{CountInversions}(R)$
 - $L = \text{Sort}(L)$
 - $R = \text{Sort}(R)$
 - $i = \text{Combine}(L, R)$
 - Retorne o número total de inversões ($i_l + i_r + i$)

- Avaliando a complexidade desse algoritmo temos:
 - $T(n) = 2T(n/2) + O(n \log n) = O(n \log^2 n)$
- Seria possível encontrar uma solução ainda mais eficiente?
 - Precisamos eliminar a etapa de ordenação explícita.
- Ideia: usar o algoritmo de intercalação para retornar a lista já ordenada à cada interação.

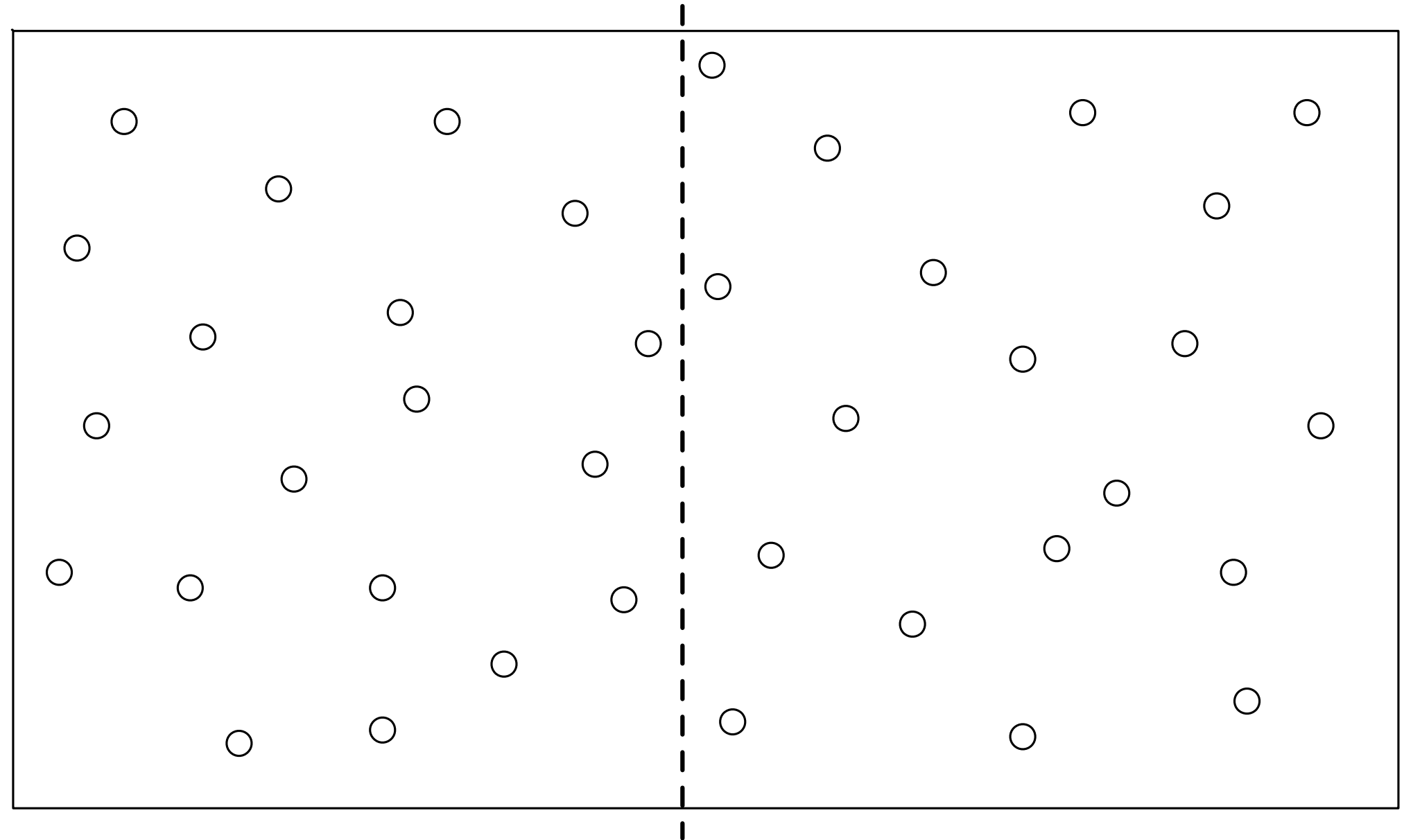
- *CountInversions(A)*:
 - Se $|A| = 1$
 - Retorne 0 inversões e a lista A
 - Divida a lista A em duas metades L e R com aproximadamente o mesmo tamanho
 - $i_l, L = \text{CountInversions}(L)$
 - $i_r, R = \text{CountInversions}(R)$
 - $i = \text{Combine}(L, R)$
 - $A = \text{Merge}(L, R)$
 - Retorne o número total de inversões $(i_l + i_r + i)$ e a lista A

- **Problema (pares mais próximos):** dado uma sequência com n pontos em um plano, encontre o par com a menor distância euclidiana.
- Solução por força bruta:
 - Teste todos os pares de pontos em $\theta(n^2)$.
- Conseguiríamos encontrar uma solução mais eficiente utilizando dividir e conquistar?

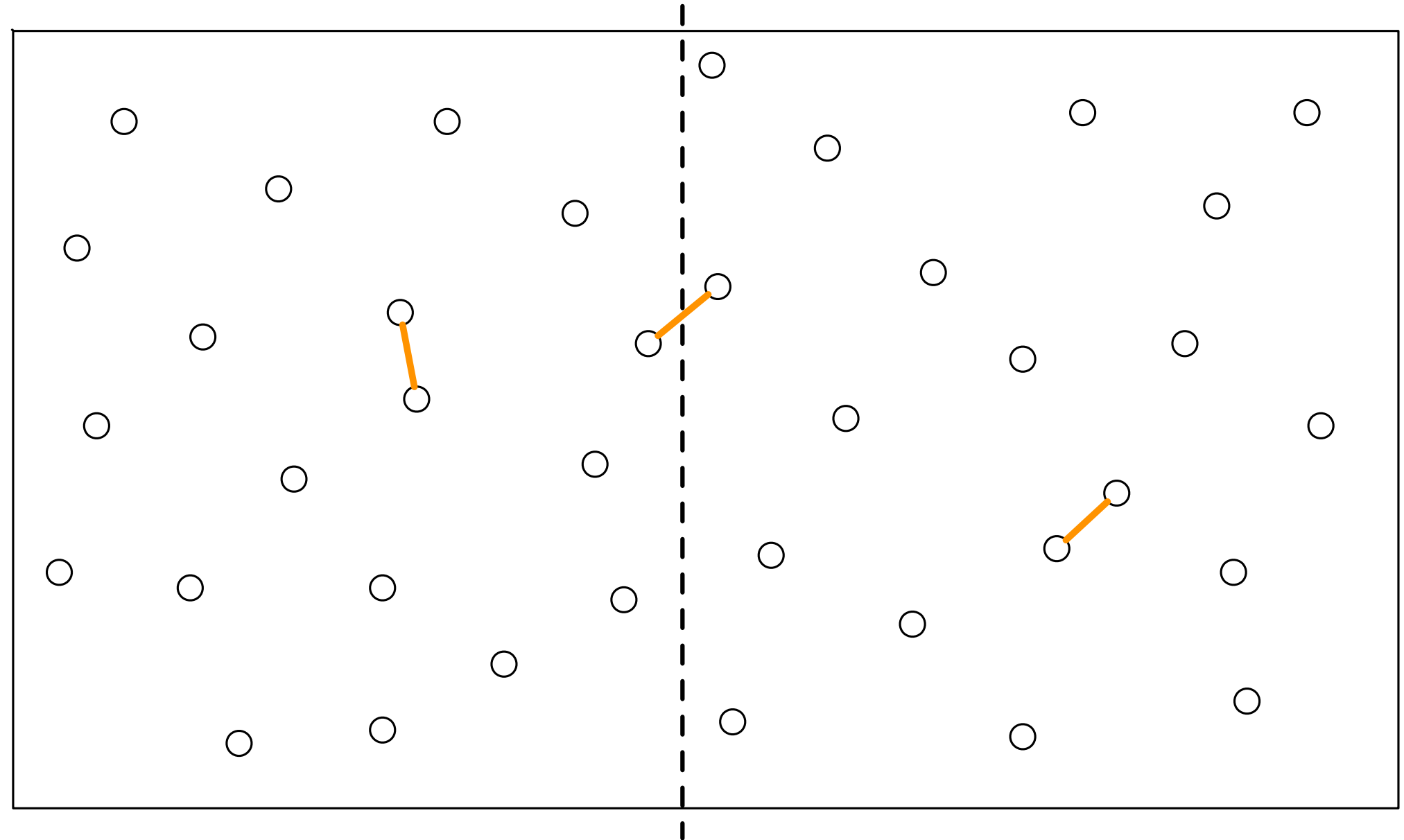


Dividir e conquistar

- Podemos dividir o plano de forma que cada lado tenha aproximadamente o mesmo número de pontos
 - Ordene os pontos pela posição no eixo x como passo inicial.
- Em seguida resolva cada lado encontrando o par mais próximo recursivamente.

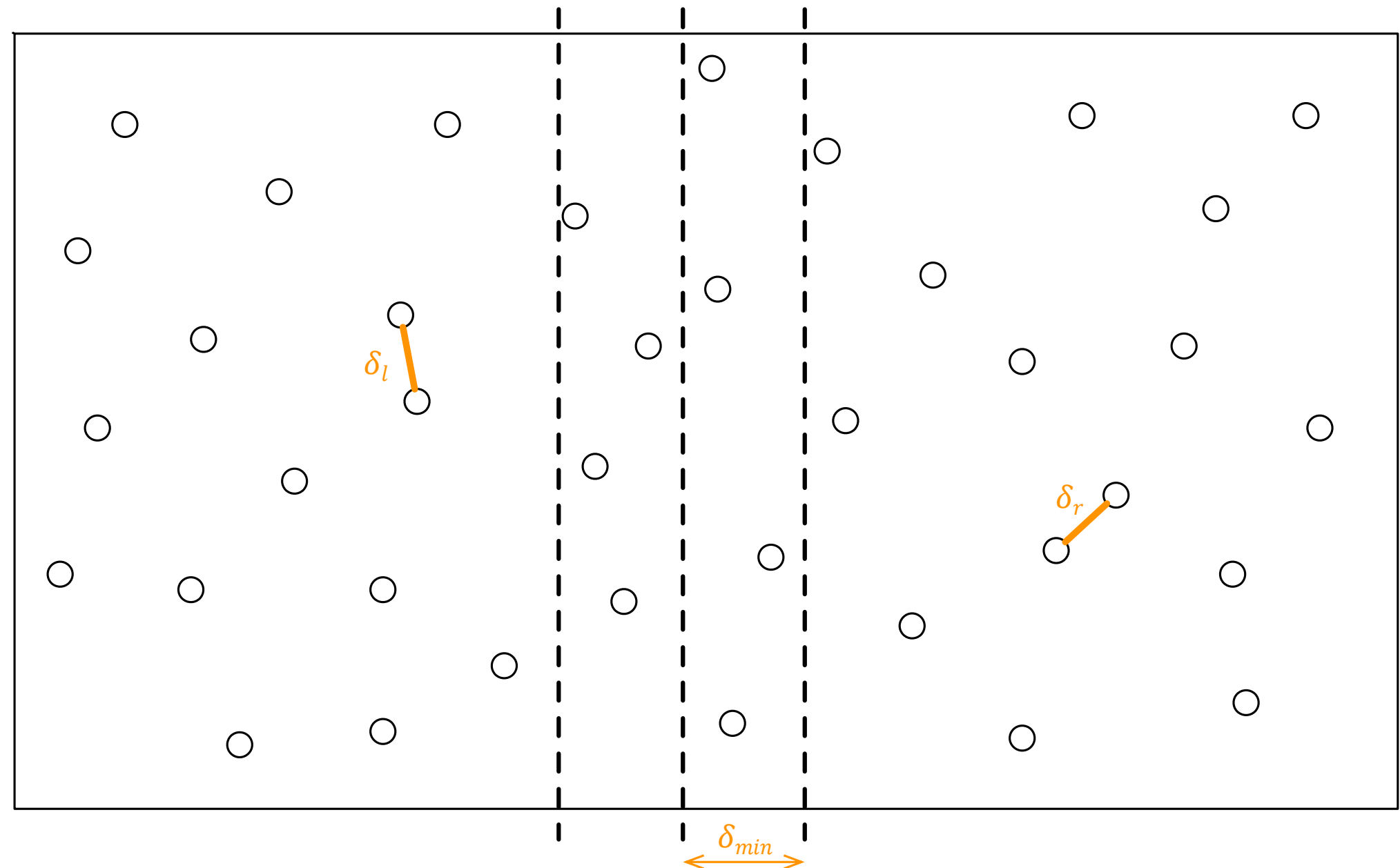


- Combine os resultados comparando:
 - O par mais próximo no lado direito.
 - O par mais próximo no lado esquerdo.
 - O par mais próximo com um ponto em cada lado.
- Esse último par parece exigir $\theta(n^2)$.

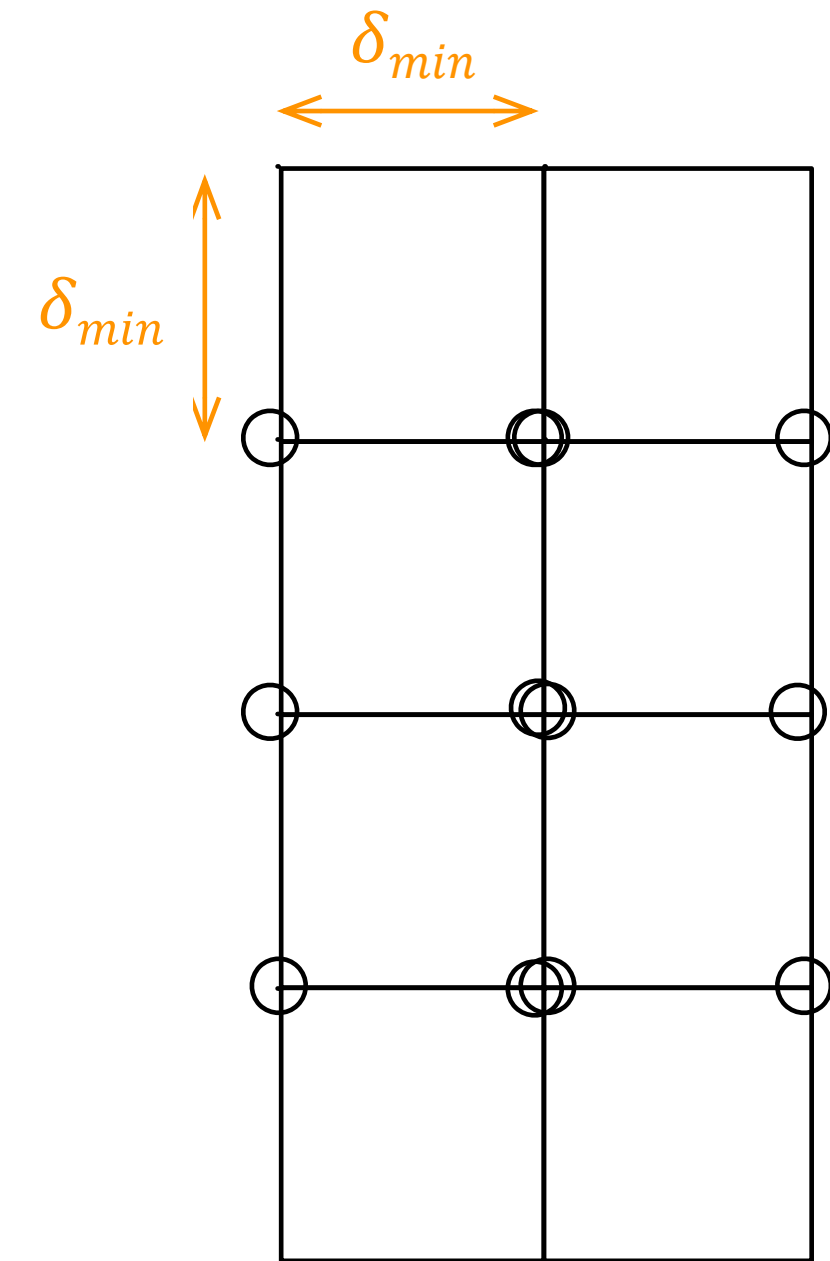


Dividir e conquistar

- Dadas as distâncias δ_l e δ_r para os pares com menor distância dos lados direito e esquerdo, procuramos um par $\delta_{min} < \min(\delta_l, \delta_r)$.
- Ideia: pesquise somente os pontos que estejam no máximo à δ_{min} da divisória.
- Ordene os pontos na faixa $2\delta_{min}$ pela posição no eixo y .



- Porque essa solução não seria $\theta(n^2)$?
- A distância entre dois pontos em cada lado é no mínimo δ_{min} .
- Dado que buscamos uma distância menor que δ_{min} , existem no máximo 11 possibilidades para cada ponto.
- Portanto o tempo de pesquisa é linear.



- *Ordene os pontos pela posição no eixo x como passo inicial.*
- *$ClosestPairs(P)$:*
 - *Se $|P| < 3$ calcule por força bruta e retorne $\delta_{\min}$.*
 - *$P_l, P_r = Split(P)$*
 - *$\delta_l = ClosestPairs(P_l)$*
 - *$\delta_r = ClosestPairs(P_r)$*
 - *$\delta_{\min} = \min(\delta_l, \delta_r)$*
 - *Selecione os P_m pontos à no máximo $\delta_{\min}$ da divisória*
 - *Ordene P_m pela posição y .*
 - *Para cada ponto p em P_m*
 - *Verifique se a distância para os 7 próximos é menor que $\delta_{\min}$*
 - *Caso seja, atualize*
 - *Retorne $\delta_{\min}$.*

- Avaliando a complexidade desse algoritmo temos:
 - $T(n) = 2T\left(\frac{n}{2}\right) + O(n) + O(n \log n) + O(n) = O(n \log^2 n)$
- Seria possível encontrar uma solução ainda mais eficiente?
 - Precisamos eliminar a etapa de ordenação explícita.
- Ideia: usar o algoritmo de intercalação para retornar a lista de pontos já ordenada pela posição y à cada interação.

- Ordene os pontos pela posição no eixo x como passo inicial.
- *ClosestPairs*(P):
 - Se $|P| < 3$ calcule por força bruta e retorne os pares ordenados e $\delta_{\min}$.
 - $P_l, P_r = \text{Split}(P)$
 - $P_l, \delta_l = \text{ClosestPairs}(P_l)$
 - $P_r, \delta_r = \text{ClosestPairs}(P_r)$
 - $\delta_{\min} = \min(\delta_l, \delta_r)$
 - $S = \text{Merge}(P_l, P_r)$
 - Selecione os P_m pontos de S à no máximo $\delta_{\min}$ da divisória
 - Para cada ponto p em P_m
 - Verifique se a distância para os 7 próximos é menor que $\delta_{\min}$
 - Caso seja, atualize
 - Retorne $S, \delta_{\min}$.

Complexidade: $\theta(n \log n)$

Programação dinâmica

Programação dinâmica

- O paradigma de **programação dinâmica** consiste em quebrar o problema em sub-problemas menores e resolvê-los de forma independente.
- É semelhante ao paradigma de divisão e conquista, no entanto os sub-problemas costumam apresentar repetições.
- Um sub-problema somente é resolvido caso não tenha sido resolvido antes
 - Caso contrário o resultado é obtido a partir de uma memória (*cache*)
 - Essa técnica é chamada de *memoization*.

- **Problema (fibonacci):** dado um inteiro $n \geq 1$ encontre F_n

- Solução recursiva (e ineficiente):

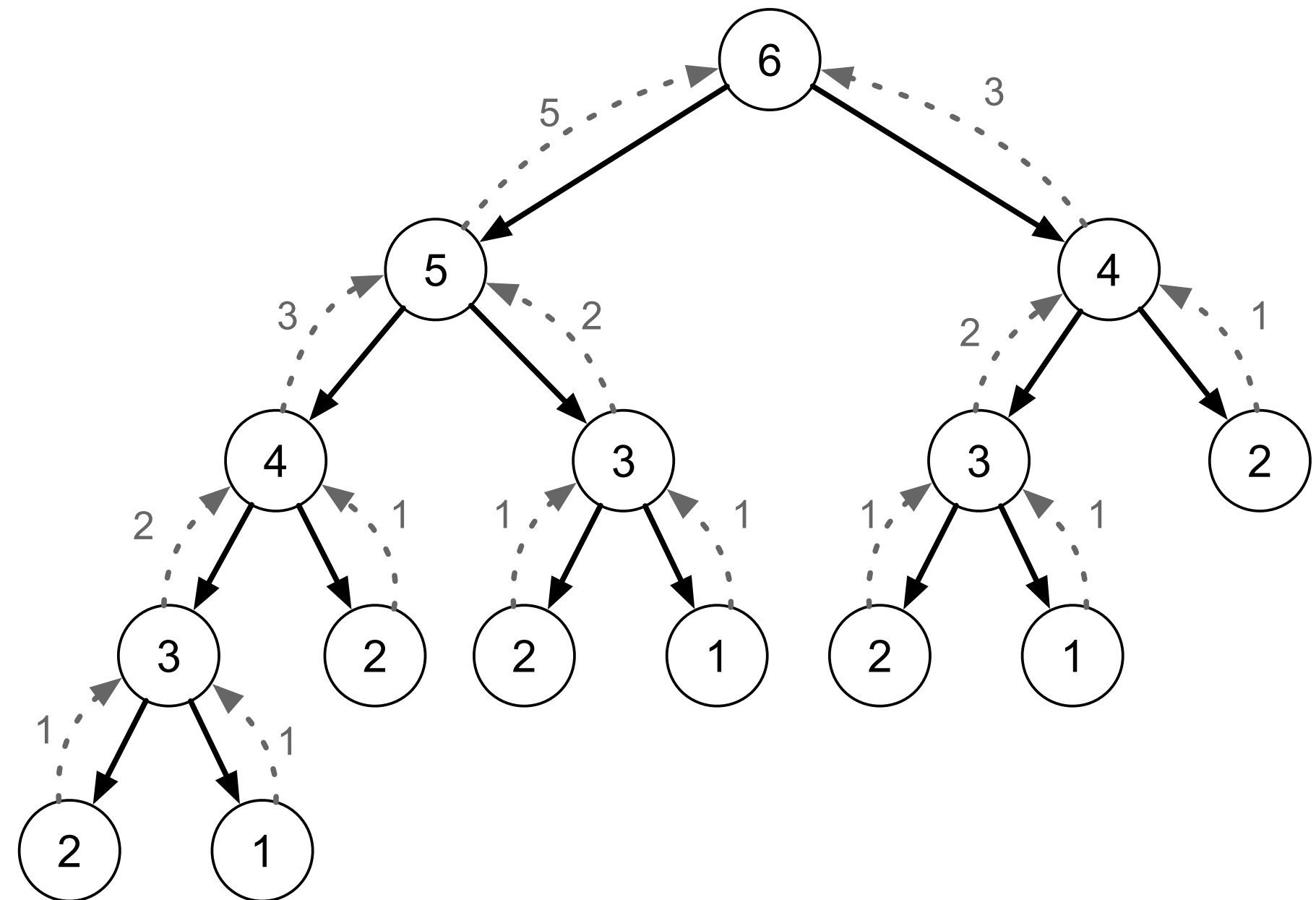
- $Fib(n)$

- Se $n \leq 2$:

- Retorna 1

- Retorna $Fib(n - 1) + Fib(n - 2)$

- Como seria a execução de $Fib(6)$?

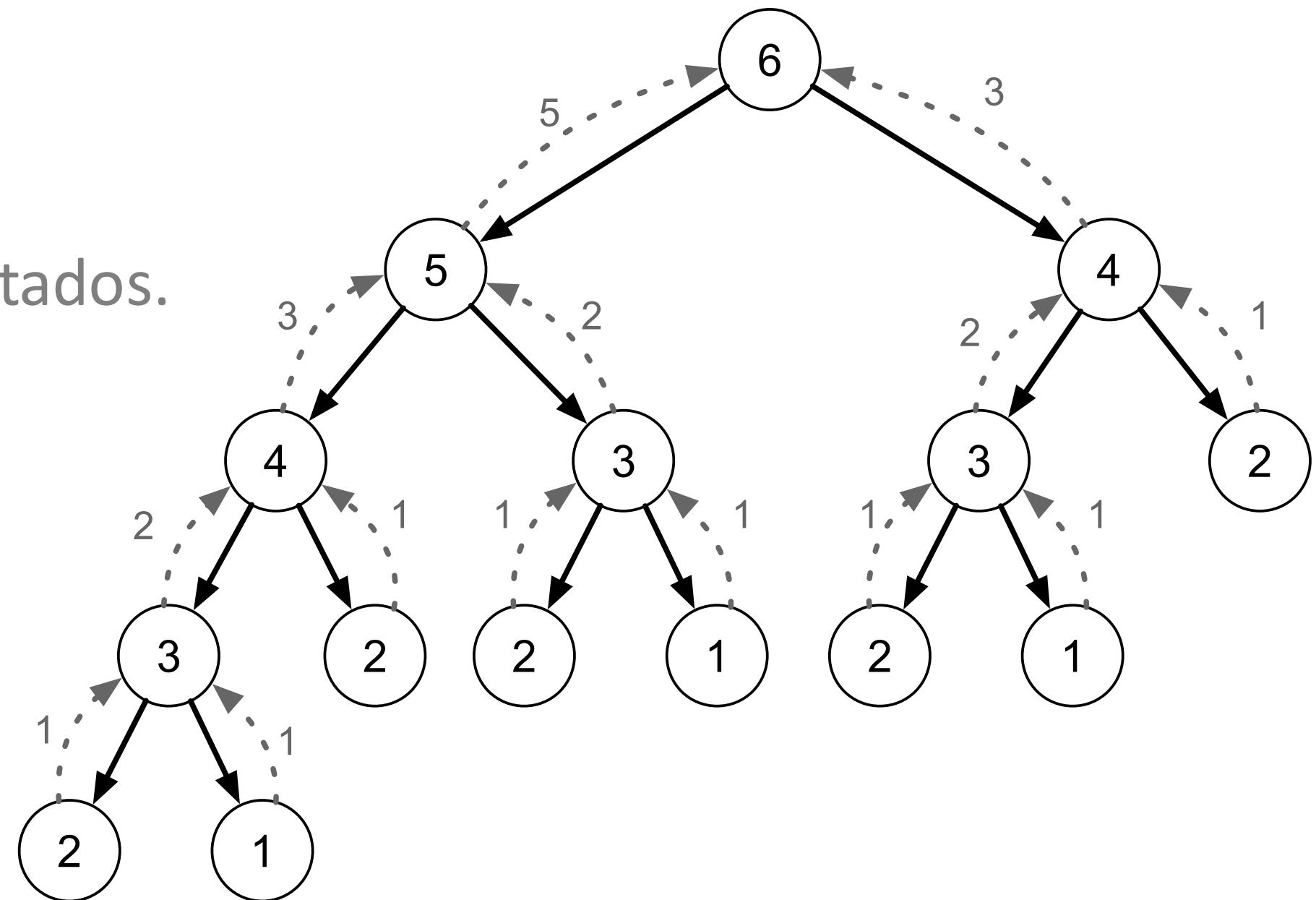


- Dado um valor n , o tempo de execução é exponencial.

- Observe que muitas sub-árvores são repetidas
 - São sub-problemas sendo re-computados.

- Existem no máximo n sub-problemas
 - Cada um é identificado pelo único parâmetro de entrada.

- Podemos utilizar um cache para reaproveitar resultados computados.



- Solução *top-down*:
 - $FibAux(n)$
 - Se $F[n] = -1$:
 - $F[n] = FibAux(n - 1) + FibAux(n - 2)$
 - Retorna $F[n]$
 - $Fib(n)$
 - Crie um vetor $F[1..n]$
 - $F[1] = 1$ e $F[2] = 1$
 - Para $i = 3$ até n :
 - $F[i] = -1$
 - Retorna $FibAux(n)$

1	2	3	4	5	6
1	1	-1	-1	-1	-1
1	1	2	-1	-1	-1
1	1	2	3	-1	-1
1	1	2	3	5	-1
1	1	2	3	5	8

Complexidade: $\theta(n)$

- Solução *bottom-up*:
 - $Fib(n)$
 - Se $n \leq 2$
 - Retorna 1
 - Crie um vetor $F[1..n]$
 - $F[1] = 1$ e $F[2] = 1$
 - Para $i = 3$ até n :
 - $F[i] = F[i - 1] + F[i - 2]$
 - Retorna $F[n]$

Complexidade: $\theta(n)$

1	2	3	4	5	6
1	1	-1	-1	-1	-1
1	1	2	-1	-1	-1
1	1	2	3	-1	-1
1	1	2	3	5	-1
1	1	2	3	5	8

- Comparando as abordagens:
 - *Top-down*
 - A solução é recursiva.
 - Somente executa a recursão caso o sub-problema não tenha sido resolvido.
 - Inicia no problema maior.
 - *Bottom-up*
 - A solução é iterativa.
 - Resolve os sub-problemas do menor para o maior.
- Em geral apresentam a mesma complexidade.

- **Problema (mochila01):** dado um conjunto de itens $I = \{1, 2, 3, \dots, n\}$ em que cada item $i \in I$ tem um peso w_i e um valor v_i , e uma mochila com capacidade de peso W , encontre o subconjunto $S \subseteq I$ tal que $\sum_{i \in S} w_i \leq W$ e $\sum_{i \in S} v_i$ seja máximo.
- Exemplo:
 - $W = 11$.
 - A escolha $\{1, 2, 4\}$ tem peso 9, valor 29 e cabe na mochila.
 - A escolha $\{3, 5\}$ tem peso 12, valor 46 e não cabe na mochila.

Item	Peso	Valor
1	1	1
2	2	6
3	5	18
4	6	22
5	7	28

- **Solução (ineficiente):** criar um algoritmo de força bruta que testa todas as possibilidades e escolhe a que cabe na mochila e apresenta maior valor
- $\text{max} = -1$
- *Para cada $S_i \subseteq I$*
 - Se $\sum_{i \in S} w_i \leq W$
 - $\text{value} = \sum_{i \in S} v_i$
 - Se $\text{value} > \text{max}$
 - $\text{max} = \text{value}$
- *Retorna max*

Complexidade: $\theta(2^n n)$

- Ideia para criar uma solução baseada em programação dinâmica:
 - Para cada item i considere a possibilidade de adicioná-lo ou não à mochila
 - Se adicionado, o valor é incrementado de v_i e a capacidade é reduzida de w_i .
 - Avalie qual o melhor valor obtido em cada caso.
 - Após considerar esse item restam $n - 1$ itens disponíveis para serem avaliados
 - Encontramos a subestrutura ótima!
- Observações:
 - Existem dois casos base: não ter mais itens e a capacidade da mochila ser zero.
 - Somente pode considerar um item na mochila se ele couber.

- Ideia geral baseada em programação dinâmica (ainda sem cache):
 - $mochila(I, v, w, W)$
 - Se $|I| = 0$ ou $W = 0$
 - Retorna 0
 - Escolha um item $i \in I$
 - Se $w_i > W$
 - Retorna $mochila(I - i, v, w, W)$
 - $value_using = v_i + mochila(I - i, v, w, W - w_i)$
 - $value_not_using = mochila(I - i, v, w, W)$
 - Retorna $\max\{value_using, value_not_using\}$

- Solução *top-down*:
- $mochila(n, v, w, W)$
 - Crie uma matriz $M[0..n][0..W]$
 - Para $i = 0$ até W :
 - $M[0][i] = 0$
 - Para $j = 1$ até n :
 - $M[j][0] = 0$
 - $M[j][i] = -1$
 - Retorna $MochilaAux(n, v, w, W)$

$W = 11$

i	w_i	v_i	0	1	2	3	...
0			0	0	0	0	...
1	1	1	0	-1	-1	-1	...
2	2	6	0	-1	-1	-1	...
3	5	18	0	-1	-1	-1	...
4	6	22	0	-1	-1	-1	...
5	7	28	0	-1	-1	-1	...

- Solução *top-down*:
- $mochilaAux(i, v, w, W)$
 - Se $M[i][W] = -1$
 - Se $w_i > W$
 - $M[i][W] = mochilaAux(i - 1, v, w, W)$
 - Caso contrário
 - $using = v_i + mochilaAux(i - 1, v, w, W - w_i)$
 - $not_using = mochilaAux(i - 1, v, w, W)$
 - $M[i][W] = \max\{using, not_using\}$
 - Retorna $M[i][W]$

- Solução *bottom-up*:
- $mochila(n, v, w, W)$
 - Crie uma matriz $M[0..n][0..W]$
 - Para $i = 0$ até W :
 - $M[0][i] = 0$
 - Para $j = 1$ até n :
 - $M[j][0] = 0$
 - Para $j = 1$ até n :
 - Para $i = 1$ até W :
 - $mochilaAux(i, j, v, w)$
 - Retorna $M[n][W]$
- $mochilaAux(i, j, v, w)$
 - Se $w_j > i$
 - $M[j][i] = M[j - 1][i]$
 - Caso contrário
 - $using = v_j + M[j - 1][i - w_j]$
 - $not_using = M[j - 1][i]$
 - $M[j][i] = \max\{using, not_using\}$

Técnicas de projeto

Programação dinâmica

$W = 11$

$W + 1$

$n + 1$

i	w_i	v_i	0	1	2	3	4	5	6	7	8	9	10	11
0			0	0	0	0	0	0	0	0	0	0	0	0
1	1	1	0	1	1	1	1	1	1	1	1	1	1	1
2	2	6	0	1	6	7	7	7	7	7	7	7	7	7
3	5	18	0	1	6	7	7	18	19	24	25	25	25	25
4	6	22	0	1	6	7	7	18	22	24	28	29	29	40
5	7	28	0	1	6	7	7	18	22	28	29	34	34	40

- Ainda precisamos definir quais itens devem ser adicionados à mochila:

- $S = \{\}, i = W$ e $j = n$
- Enquanto $j \geq 1$
 - Se $M[j][i] = M[j - 1][i - w_j] + v_j$
 - $S = S \cup \{j\}$
 - $i = i - w_j$
 - $j = j - 1$
- Retorna S

i	w_i	v_i	0	1	2	3	4	5	6	7	8	9	10	11
0			0	0	0	0	0	0	0	0	0	0	0	0
1	1	1	0	1	1	1	1	1	1	1	1	1	1	1
2	2	6	0	1	6	7	7	7	7	7	7	7	7	7
3	5	18	0	1	6	7	7	18	19	24	25	25	25	25
4	6	22	0	1	6	7	7	18	22	24	28	29	29	40
5	7	28	0	1	6	7	7	18	22	28	29	34	34	40

- A complexidade do algoritmo é $\theta(nW)$.
- Essa complexidade não é polinomial!
- W não é um tamanho, é um valor
 - Exige $\log W$ bits para ser representado.
 - Outra forma de escrever a complexidade seria $\theta(n 2^{\log W})$
- Portanto, o algoritmo é pseudo-polinomial.

Exercícios

- **Exercício 1:** Dadas duas *strings*, encontre o comprimento da maior subsequência comum entre elas.
- **Exercício 2:** Dado um valor v e uma lista de denominações de moedas (de um sistema canônico), encontre o número mínimo de moedas para formar v .
- **Exercício 3:** Dado um array A com n elementos, encontre simultaneamente o maior e o menor elemento usando o menor número possível de comparações.

Exercícios

- **Exercício 1:** Dadas duas *strings*, encontre o comprimento da maior subsequência comum entre elas.

```
LCS (s1[1..p], s2[1..q])  
  Para i = 0 até p:  
    m[i][0] ← 0  
  
  Para j = 0 até q:  
    m[0][j] ← 0  
  
  Para i = 1 até p:  
    Para j = 1 até q:  
      Se s1[i] == s2[j]:  
        m[i][j] ← m[i-1][j-1] + 1  
      Caso contrário  
        m[i][j] ← MAX(m[i-1][j], m[i][j-1])  
  
  Retornar m[p][q]
```

Exercícios

- **Exercício 2:** Dado um valor v e uma lista de denominações de moedas (de um sistema canônico), encontre o número mínimo de moedas para formar v .

```
TROCO_GULOSO(valor, moedas[1..n])
  ordenar(moedas, DESC)

  quantidade_total ← 0
  resultado ← []

  Para moeda em moedas:
    Se valor >= moeda:
      quantidade ← valor / moeda
      quantidade_total ← quantidade_total + quantidade
      valor ← valor % moeda

      Para i = 1 até quantidade:
        resultado.append(moeda)

  Se valor > 0:
    Retornar "Não é possível formar o valor"
  Caso contrário
    Retornar quantidade_total, resultado
```


Exercícios

- **Exercício 3:** Dado um array A com n elementos, encontre simultaneamente o maior e o menor elemento usando o menor número possível de comparações.

```
MAX_MIN(A[1..n], esquerda, direita)
    Se esquerda == direita:
        Retornar A[esquerda], A[esquerda]

    Se direita == esquerda + 1:
        Se A[esquerda] < A[direita]:
            Retornar A[direita], A[esquerda]
        Caso contrário
            Retornar A[esquerda], A[direita]

    meio ← (esquerda + direita) / 2
    max1, min1 ← MAX_MIN(A, esquerda, meio)
    max2, min2 ← MAX_MIN(A, meio + 1, direita)

    max_final ← MAX(max1, max2)
    min_final ← MIN(min1, min2)

    Retornar max_final, min_final
```

Perguntas?

`thiago.p.araujo@fgv.br`